

Birla Institute of Technology and Science, Pilani

Work Integrated Learning Programmes Division

M. Tech. in AI & ML

I Semester 2025-2026

Mid-Semester Test

(EC3 - Regular)

Course Number	AIMLCZG511
Course Name	DEEP NEURAL NETWORK
Nature of Exam	Open Book
Weight-age for grading	40
Total Marks	120

# Pages	31
# Questions	8

-
1. A DFNN with two hidden layers classifies medical X-ray images into 10 categories (Normal, Pneumonia, Cardiomegaly, etc.).
 - (a) A neuron in Hidden Layer 1 receives input $\mathbf{x} = [0.8, -0.5, 0.2]$, weights $\mathbf{w} = [0.4, 0.1, -0.2]$, bias $b = 0.5$.
 - i. Compute activation by applying ReLU. Compare Softmax vs ReLU outputs for this 10-class problem. Which is appropriate and why? (4 marks)
 - ii. Define activation functions for Hidden Layer 1 (ReLU), Hidden Layer 2 (Leaky ReLU), and Output Layer (Softmax) with mathematical formulas. (3 marks)
 - (b)
 - i. Explain why a Perceptron cannot solve this 10-class task, referencing linear separability, decision boundaries, and the XOR problem as analogy. (4 marks)
 - ii. During DFNN training, gradients near input layers vanish. Identify the cause, explain the impact on learning, and recommend TWO architectural fixes with justification. (4 marks)
 2. A telecom company predicts subscriber churn using call duration, data usage, complaints, and contract type.
 - (a)
 - i. Identify the output activation and loss function for binary churn prediction. Provide formulas, justify their pairing, and explain what a classification threshold of 0.5 represents in this context. (4 marks)
 - ii. A logistic regression model achieves 88% accuracy but 54% recall for churners. Identify TWO architectural reasons why a DFNN captures churn signals better than logistic regression. (3 marks)
 - (b) Design a complete DFNN for churn prediction.
 - i. Justify the choice of ReLU for Hidden Layer 1 and Leaky ReLU for Hidden Layer 2, addressing computational efficiency and the dying ReLU problem. (3 marks)
 - ii. Specify the complete architecture: input dimension, hidden layers, neurons per layer, activations, loss function, and output configuration. Justify each choice for churn prediction. (5 marks)
 3. A hospital develops a skin cancer detection system using 8,000 dermoscopic images ($512 \times 512 \times 3$) across 7 lesion types. Melanoma: 600 images, benign nevi: 3,200. Pre-trained ResNet-50 achieves 92% overall accuracy but 68% melanoma sensitivity. Constraints are 8GB GPU, <500ms inference.
 - (a)
 - i. Should the team use transfer learning or train from scratch? Provide THREE dataset-specific reasons. The ResNet-50 final FC layer is replaced with 2048 $\rightarrow$

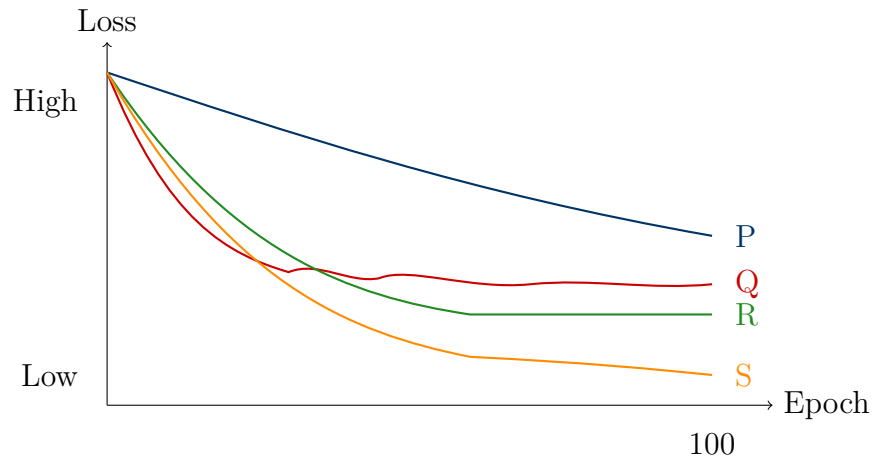
- 512 $\rightarrow$ 7 with Dropout ($p = 0.5$). Calculate the parameters removed from the original FC layer, the parameters added by the new classifier (include biases), and the net parameter change. (5 marks)
- ii. A wildlife CNN ($224 \times 224 \times 3$ input, 10 classes) has Conv(32, 5×5 , $s=1$, $p=2$) $\rightarrow$ MaxPool(2×2 , $s=2$) $\rightarrow$ Conv(64, 3×3 , $s=1$, $p=1$) $\rightarrow$ MaxPool $\rightarrow$ Conv(128, 3×3 , $s=1$, $p=1$) $\rightarrow$ MaxPool $\rightarrow$ Flatten $\rightarrow$ FC(100352 $\rightarrow$ 256) $\rightarrow$ FC(256 $\rightarrow$ 10). Calculate total parameters (with biases) for Layer 1 (first Conv) and Layer 8 (first FC). (5 marks)
- (b) i. The wildlife team adds infrared; input becomes $224 \times 224 \times 4$. Calculate new parameter count of Layer 1 and percentage increase. Do Layer 3 or Layer 8 parameters change? Explain why or why not. Layer 8 contains 99.2% of parameters. Propose ONE solution from Global Average Pooling ($28 \times 28 \times 128 \rightarrow 128 \rightarrow$ FC), additional Conv to reduce spatial size, or 1×1 Conv to reduce channels. Calculate the new FC pathway parameter count and state ONE advantage and ONE disadvantage for edge deployment. (6 marks)
- ii. The ResNet-50 achieves 89% validation and 94% training accuracy on full images but drops to 71% on 128×128 crops. Explain TWO CNN architectural reasons for this degradation. A researcher claims adding the infrared channel increases the receptive field at Layer 5. Is this correct? Justify with reference to receptive field computation. (4 marks)
4. An industrial edge device performs real-time anomaly detection on factory motor sensor streams (vibration, temperature, current). Data arrives continuously at high frequency; anomalies stem from recent patterns; device has limited memory, compute, power.
- (a) Given input dimension $d = 10$, hidden dimension $h = 20$:
- i. Compute total trainable parameters (exclude output layer) for Vanilla RNN and GRU. (2 marks)
- ii. Select the most appropriate model (Vanilla RNN, GRU, or LSTM) for the edge device scenario. Justify addressing memory, compute constraints, and anomaly signal characteristics. (5 marks)
- (b) Model a single time step with $\mathbf{x}_t = [2, 1]^\top$, $\mathbf{h}_{t-1} = [0.5, -0.5]^\top$, $d = 2$, $h = 2$, all biases zero.
- i. For Vanilla RNN with $W_{hh} = \begin{bmatrix} 0.6 & 0 \\ 0 & 0.6 \end{bmatrix}$, $W_{hx} = \begin{bmatrix} 0.4 & 0 \\ 0 & 0.4 \end{bmatrix}$: compute pre-activation and $\mathbf{h}_t$. For GRU with $W_z = I$, $U_z = 0$, $W_h = 0.5I$, $U_h = 0$, compute $\mathbf{z}_t$, $\tilde{\mathbf{h}}_t$, $\mathbf{h}_t$. State ONE numerical reason why GRU is more conservative. (5 marks)
- ii. A GRU models a 3-step sequence with $\mathbf{h}_t = z_t \mathbf{h}_{t-1} + (1 - z_t) \tilde{\mathbf{h}}_t$. Given $\partial L / \partial \mathbf{h}_3 = 1$, $z_t = 0.9$ for $t = 2, 3$, candidate path ignored: compute $\partial L / \partial \mathbf{h}_1$. Explain how the update gate mitigates vanishing gradients and identify ONE scenario where GRU still struggles. Under what operational constraints would RNN be preferred over Transformer for factory anomaly detection? Justify with complexity. (3 marks)
5. (a) In a product recommendation system, users browse sequentially: Phones $\rightarrow$ Bags $\rightarrow$ Headset. Self-attention identifies which past items signal the next recommendation. For the third position (Headset), given $\mathbf{q}_3 = [2, 7]$, keys $\mathbf{k}_1 = [3, 1]$, $\mathbf{k}_2 = [3, 1]$, $\mathbf{k}_3 = [7, 3]$, and values $\mathbf{v}_1 = [3, 1]$, $\mathbf{v}_2 = [3, 1]$, $\mathbf{v}_3 = [8, 3]$:
- i. Using scaled dot-product attention, compute raw scores, apply softmax, obtain context output vector. (3 marks)
- ii. The deployed model recommends exactly what users bought in training but fails on future purchases in production. Identify the missing attention mechanism during training and justify. (4 marks)
- (b) Electronic health records contain patient event sequences: Blood Test $\rightarrow$ X-Ray $\rightarrow$

Medication. Additive attention identifies which past medical events most influence the next clinical decision. For the third position (Medication), given query $\mathbf{q}_3 = [2, 1]$, keys $\mathbf{k}_1 = [1, 0]$, $\mathbf{k}_2 = [0, 1]$, $\mathbf{k}_3 = [2, 1]$, and values $\mathbf{v}_1 = [1, 0]$, $\mathbf{v}_2 = [1, 1]$, $\mathbf{v}_3 = [3, 1]$:

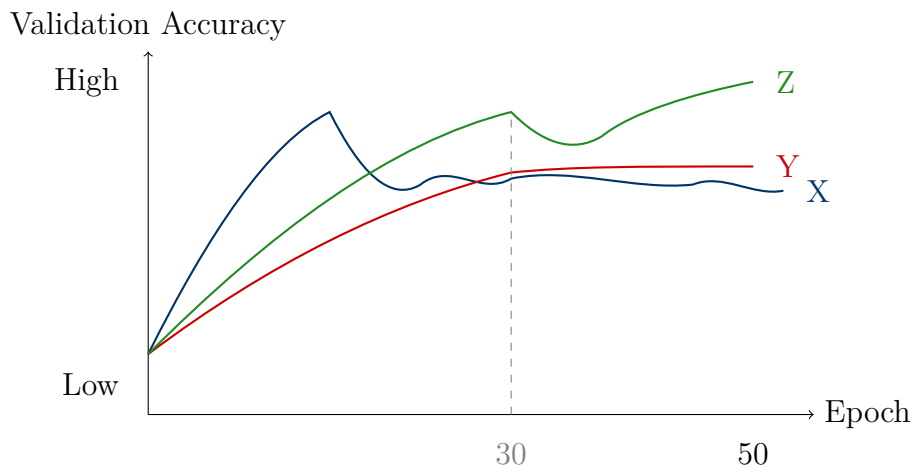
- i. Using additive attention with tanh scoring, compute alignment scores, softmax weights, and context output. Explain why additive attention is used instead of scaled dot-product attention for this electronic health records scenario. (5 marks)
 - ii. The clinical team decides to add textual notes (doctor observations, patient symptoms) to the EHR sequence and debates between single-head versus multi-head attention ($d_{model} = 512$, $h = 8$ heads). Compute each head's Q/K/V dimension. Calculate the increase in parameter count when moving from single-head to multi-head attention. The single-head model attends only to recent events, ignoring critical older diagnostic signals. Explain how multi-head attention resolves this and describe what different heads might learn in the clinical context. Identify ONE limitation for resource-constrained deployment. (3 marks)
6. A healthcare NLP team builds a clinical report summarisation system processing long discharge notes into concise summaries.
- (a)
 - i. The team removes positional encoding to simplify architecture. Explain what positional encoding does and describe the specific failure mode in clinical text without it. (5 marks)
 - ii. Match encoder-only, decoder-only, and full encoder-decoder architectures to the following three tasks: clinical named entity recognition, discharge note generation, and medical report summarisation. Justify each match. (5 marks)
 - (b) The encoder has $d_{model} = 512$, 8 heads, $d_{ff} = 2048$, $L = 128$ tokens.
 - i. For one encoder layer, trace tensor dimensions at each stage: input $\rightarrow$ multi-head attention output $\rightarrow$ add & norm $\rightarrow$ feed-forward output $\rightarrow$ add & norm. Then compute total trainable parameters in this encoder layer, accounting for multi-head attention (Q/K/V projections and output projection) and the feed-forward sublayer. (6 marks)
 - ii. The team must choose between BERT-style, GPT-style, or T5-style architectures for the summarisation task, given 50,000 annotated note-summary pairs and four A100 GPUs. For each architecture, compare the pre-training objective, architectural form (encoder/decoder/both), and suitability for summarisation. Clinical notes can reach 2,048 tokens, and self-attention has $O(L^2)$ complexity with respect to sequence length. Explain why this creates a computational bottleneck and propose ONE mitigation strategy along with its trade-off. (4 marks)

7. A team trains a DNN for financial fraud detection.

- (a) Training loss curves for four optimizers (SGD, SGD+Momentum, RMSprop, Adam) over 100 epochs are shown below:



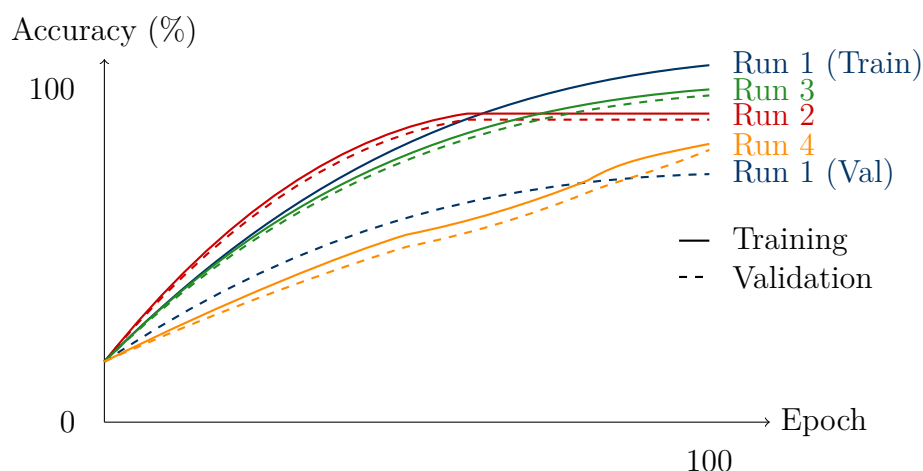
- Match each curve (P,Q,R, S) to its optimizer. Justify each match in one sentence. (4 marks)
 - After epoch 60 using Adam, validation loss rises while training loss falls. Identify this phenomenon and suggest ONE optimizer-level intervention (not architectural). (1 mark)
- (b) Three learning rate schedules applied to the same Adam optimizer produce the following validation accuracy curves over 50 epochs:



- Identify the schedule type for X, Y, and Z from the following options: constant high learning rate, constant low learning rate, or cosine annealing with warm restarts. Justify each match. (3 marks)
- Explain why Schedule Z exhibits a brief dip at epoch 30 and how it recovers to achieve the highest final accuracy. (2 marks)

8. A team trains a DNN for financial fraud detection.

- (a) Four regularization experiments produce the following training and validation accuracy curves over 100 epochs:



- i. Based on the training and validation accuracy curves shown above, diagnose the regularisation condition for each run. Classify each as overfitting, underfitting, well-regularised, or over-regularised, and justify your diagnosis using the gap between training and validation curves and their convergence behavior. (4 marks)
 - ii. Run 1 shows a large train-validation gap that persists throughout training. Propose ONE specific regularisation techniques to reduce this gap and explain the mechanism by which each technique addresses the overfitting observed in Run 1. (1 mark)
- (b)
- i. The team adds Batch Normalisation to Run 4. Predict the effect on training stability and convergence speed, explaining the mechanism. (3 marks)
 - ii. Compare Dropout vs Batch Normalisation in terms of when would you use each technique, and can they be combined? Justify with architectural reasoning. (2 marks)

Question 1: Medical X-ray Classification System

(a) Neuron Inference and Activation Functions [7 marks]

Part (i): ReLU Inference Computation [2 marks]

Given: $\mathbf{x} = [0.8, -0.5, 0.2]$, $\mathbf{w} = [0.4, 0.1, -0.2]$, $b = 0.5$

Step 1: Compute net input

$$z = \mathbf{w}^\top \mathbf{x} + b = (0.4)(0.8) + (0.1)(-0.5) + (-0.2)(0.2) + 0.5 = 0.73$$

Step 2: Apply ReLU activation

$$\alpha = \text{ReLU}(z) = \max(0, z) = \max(0, 0.73) = 0.73$$

Component	Full Credit (2)	Partial (1.5)	Partial (1)	No Credit (0)
Net input calculation (1 mark)	Correct $z = 0.73$ with all steps shown	Correct for formula but minor arithmetic error	Correct approach but significant error	Wrong formula or missing
ReLU application (1 mark)	Correct $\alpha = 0.73$ with ReLU formula stated	Correct result but formula not stated	Applied wrong activation or error	Missing or completely wrong

Part (ii): Softmax Formula and Justification [2 marks]

Softmax Formula:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{10} e^{z_j}} \quad \text{for } i = 1, 2, \dots, 10$$

Justification: Softmax is appropriate because it converts 10 raw logit scores into a valid probability distribution where all outputs sum to 1.0, ensuring each class receives a probability value between 0 and 1, which is required for multi-class classification tasks.

Alternative Acceptable Justifications:

- Softmax emphasizes the largest logit (winner-take-all behavior) making class predictions clearer
- Provides calibrated probabilities suitable for cross-entropy loss
- Allows probabilistic interpretation for medical diagnosis confidence

Component	Full Credit (2)	Partial (1.5)	Partial (1)	No Credit (0)
Formula (1 mark)	Correct softmax formula with summation over all 10 classes	Minor notation error but concept clear	Incomplete formula (e.g., missing denominator)	Wrong formula or missing
Justification (1 mark)	Clear explanation of probability distribution requirement	Mentions normalization but lacks detail	Vague statement about classification	No justification or wrong

Part (iii): Activation Function Definitions [3 marks]

Hidden Layer 1 - ReLU:

$$f(z) = \max(0, z) = \begin{cases} z & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$$

Hidden Layer 2 - Leaky ReLU:

$$f(z) = \max(\alpha z, z) = \begin{cases} z & \text{if } z > 0 \\ \alpha z & \text{if } z \leq 0 \end{cases} \quad \text{where } \alpha = 0.01$$

Output Layer - Softmax:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

where $K = 10$ (number of classes)

Keywords for Partial Credit:

- ReLU: "computationally cheap", "no exponentials", "sparse activation", "fast gradient"
- Dying ReLU: "neurons stop learning", "zero gradient for negatives", "permanently inactive"
- Leaky ReLU: "small slope for negatives", "alpha parameter", "keeps neurons alive"

Component	Full Credit (3)	Partial (2-2.5)	Partial (1-1.5)	No Credit (0-0.5)
ReLU (1 mark)	Correct formula with max or piecewise notation	Minor notation issue but concept clear	Incomplete formula	Wrong or missing
Leaky ReLU (1 mark)	Correct formula with α parameter for negative values	Shows negative handling but missing α	Confuses with standard ReLU	Wrong or missing
Softmax (1 mark)	Correct formula with exponentials and summation	Minor notation error	Missing key component (exp or sum)	Wrong or missing

(b) Perceptron Limitations and Vanishing Gradients [8 marks]

Part (i): Perceptron Cannot Solve 10-class Task [4 marks]

- Linear separability:** Perceptron creates only linear hyperplane boundaries; 10-class medical images require non-linear decision regions as conditions share visual features
- Single boundary limitation:** One linear boundary per neuron cannot model complex relationships between visually similar conditions (e.g., pneumonia subtypes)
- XOR analogy:** Just as XOR needs non-linear boundary, medical classification requires hierarchical non-linear feature combinations that Perceptron cannot learn
- No feature learning:** Without hidden layers, cannot extract intermediate representations (edges, textures, shapes)—direct pixel-to-class mapping insufficient for diagnosis

Keywords for Partial Credit:

- "Linear separability", "hyperplane", "decision boundary"
- "XOR", "non-linear", "cannot learn"
- "Hidden layers needed", "feature hierarchy"
- "Complex patterns", "high-dimensional space"

Part (ii): Vanishing Gradients - Cause, Impact, and Fixes [4 marks]

Cause (1 mark):

- Saturating activations (sigmoid/tanh) have bounded derivatives (max 0.25 for sigmoid)
- Backpropagation multiplies derivatives across layers via chain rule
- Repeated multiplication causes exponential gradient decay through deep networks

Component	Excellent (4)	Good (2-3)	Poor (0-1)
Linear separability explanation (1.5 marks)	Clear explanation of linear boundary limitation with reference to multi-class complexity	Mentions linear separability but lacks depth	Vague or missing
Decision boundary analysis (1 mark)	Explains single boundary per neuron limitation	Mentions boundaries but incomplete	Missing or wrong
XOR analogy (1 mark)	Correctly relates XOR non-linear separability to medical task	Mentions XOR but weak connection	Missing or irrelevant
Feature learning limitation (0.5 marks)	Explains lack of hierarchical feature extraction	Mentions features superficially	Missing

Impact on Learning (1 mark):

- Gradients near input layers approach zero: $\Delta w = -\eta \frac{\partial L}{\partial w} \approx 0$
- Early layers stop learning—cannot extract low-level features
- Network becomes effectively shallow; only final layers train properly

Fix 1: Use ReLU Activations (1 mark):

- ReLU derivative = 1 for positive inputs (no saturation)
- Prevents gradient decay—gradients flow unchanged through active neurons
- Maintains gradient magnitude across layers

Fix 2: Add Residual Connections (1 mark):

- Skip connections: $y = F(x) + x$ create gradient highways
- Gradients flow directly backward through identity mappings, bypassing layers
- Ensures deep layers receive strong learning signals

Alternative Acceptable Fixes:

- Batch Normalization - stabilizes gradients by normalizing layer inputs
- Proper weight initialization (Xavier/He) - prevents initial gradient issues
- Layer Normalization - alternative to batch norm for smaller batches

Component	Full Credit	Partial	No Credit
Cause identification (1 mark)	Explains saturating activations + chain rule multiplication causing decay	Mentions gradient decay but lacks mechanism	Missing or wrong
Impact on learning (1 mark)	Explains negligible weight updates in early layers	Vague statement about "slow learning"	Missing or wrong
Fix 1 with justification (1 mark)	Names fix + explains mechanism clearly	Names fix but weak explanation	Just names technique
Fix 2 with justification (1 mark)	Names fix + explains mechanism clearly	Names fix but weak explanation	Just names technique

Keywords for Partial Credit:

- Cause: "sigmoid saturation", "chain rule", "gradient multiplication", "exponential decay"
- Impact: "early layers don't learn", "small weight updates", "shallow effective depth"
- Fixes: "ReLU", "residual connections", "skip connections", "batch normalization", "proper initialization"

Question 2: Telecom Churn Prediction

(a) Binary Classification Fundamentals [7 marks]

Part (i): Output Activation and Loss Function [4 marks]

Output Activation - Sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Output range: $(0, 1)$, interpreted as churn probability.

Loss Function - Binary Cross-Entropy:

$$L(y, \hat{y}) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

where $y \in \{0, 1\}$ is true label, $\hat{y} \in (0, 1)$ is predicted probability.

Why Paired:

- Gradient simplification: $\frac{\partial L}{\partial z} = \hat{y} - y$ (clean, stable training)
- Probabilistic interpretation: sigmoid outputs valid probabilities, BCE measures distribution divergence
- Convex optimization: together create convex loss surface for gradient descent

Threshold (0.5) Explanation: If $\hat{y} \geq 0.5$, predict churn; else no churn. Represents decision boundary—customers above 50% probability flagged for retention. Adjustable based on business needs: lower threshold (0.3) catches more churners but increases false alarms; higher threshold (0.7) targets only high-confidence cases.

Component	Full (4)	Partial (2-3)	Poor (0-1)
Sigmoid + BCE formulas (1 mark)	Both correct	Minor errors	Missing/wrong
Pairing justification (2 marks)	Gradient + probabilistic explanation	One reason clear	Vague
Threshold explanation (1 mark)	Clear with telecom context	Generic explanation	Missing

Keywords for Partial Credit:

- "Sigmoid probability $[0,1]$ ", "cross-entropy", "log loss", "gradient simplifies", "threshold 0.5", "decision boundary"

Part (ii): Logistic Regression vs DFNN [3 marks]

Reason 1: Non-linear Feature Interactions (1.5 marks)

- Logistic regression: only linear relationships, requires manual feature engineering
- DFNN hidden layers: automatically learn complex non-linear patterns
- Example: high usage + high complaints $\rightarrow$ churn (non-linear interaction DFNN captures)

Reason 2: Hierarchical Feature Representations (1.5 marks)

- Logistic regression: operates on raw features directly
- DFNN: builds hierarchical abstractions (Layer 1: usage bins $\rightarrow$ Layer 2: customer profiles)
- Discovers latent behavioral segments predicting churn

Alternative Reasons:

- More parameters $\rightarrow$ model complex decision boundaries
- Automatic feature learning vs manual engineering

Keywords: "Non-linear", "feature interactions", "hidden layers", "hierarchical", "representation learning"

Component	Full (3)	Poor (0-1)
Two architectural reasons with churn context (3 marks)	Both clear with examples	Vague/generic

(b) DFNN Architecture Design [8 marks]

Part (i): Activation Justification [3 marks]

Hidden Layer 1 - ReLU (1.5 marks):

- Simple computation: $f(z) = \max(0, z)$ (no exponentials)
- Fast gradient: derivative 0 or 1
- Sparse activation: 50% neurons inactive
- No saturation for positive values—prevents vanishing gradients

Hidden Layer 2 - Leaky ReLU (1.5 marks):

- Dying ReLU problem: neurons with negative output stop learning (gradient = 0)
- Leaky ReLU: $f(z) = \max(\alpha z, z)$ where $\alpha = 0.01$ allows small negative gradients
- Keeps all neurons alive throughout training
- Critical for deeper layers—prevents loss of churn pattern capacity

Component	Full	Poor
ReLU efficiency (1.5 marks)	Multiple aspects explained	Vague
Leaky ReLU dying problem (1.5 marks)	Problem + solution clear	Incomplete

Keywords: "Computationally cheap", "no exponentials", "dying ReLU", "neurons stop learning", "small slope negatives"

Part (ii): Complete Architecture [5 marks]

Architecture Specification:

Input Layer (0.5 marks):

- 4 neurons (call duration, data usage, complaints, contract type)
- One-hot encode categorical features (e.g., contract type $\rightarrow$ 3 binary features)

Hidden Layers (1.5 marks):

- Layer 1: 16-32 neurons, ReLU (wider for diverse patterns)
- Layer 2: 8-16 neurons, Leaky ReLU (bottleneck for high-level features)
- Justification: decreasing sizes, sufficient depth without over-complication

Output Layer (0.5 marks):

- 1 neuron, Sigmoid activation
- Outputs churn probability $\in (0, 1)$

Loss Function (0.5 marks):

- Binary Cross-Entropy (paired with sigmoid)
- Can weight for class imbalance

Component	Excellent	Good	Poor
Input (0.5 marks)	Correct with encoding	Basic	Wrong
Hidden layers (1.5 marks)	Sizes + activations + reasoning	Specified, weak justification	Unreasonable
Output (0.5 marks)	1 neuron, sigmoid, justified	Correct, minimal	Wrong
Loss (0.5 marks)	BCE with justification	BCE stated	Wrong
Overall justification (2 marks)	Churn-specific context throughout	Generic justifications	Missing

Keywords: "Input 4", "hidden layers", "ReLU/Leaky ReLU", "sigmoid output", "BCE", "churn probability"

Question 3: CNN Architecture Validation

(a) CNN Dimension and Parameter Analysis [10 marks]

Context: Defect classification system processes RGB images ($128 \times 128 \times 3$) into 4 categories (scratch, crack, dent, no defect). Architecture includes 3 Conv layers, 3 Pooling layers, Flatten, and FC layers.

Part (i): Output Dimension Calculation [5 marks]

Convolution Output Formula:

$$\text{Output size} = \left\lfloor \frac{\text{Input size} + 2 \times \text{Padding} - \text{Kernel size}}{\text{Stride}} \right\rfloor + 1$$

Detailed Calculation for Conv Layer 1 (2 marks):

Given (from architecture diagram): Input $128 \times 128 \times 3$, filters = 32, kernel = 5×5 , stride = 1, padding = 2

Height/Width:

$$\text{Output} = \left\lfloor \frac{128 + 2(2) - 5}{1} \right\rfloor + 1 = 128$$

Depth: 32 filters $\rightarrow$ 32 channels

Conv Layer 1 Output: $128 \times 128 \times 32$

Verification for Remaining Layers (3 marks):

Expected dimensions (assuming standard architecture):

- **Pool1:** MaxPool 2×2 , stride 2 $\rightarrow 64 \times 64 \times 32$
- **Conv2:** 64 filters, 3×3 , stride 1, padding 1 $\rightarrow 64 \times 64 \times 64$
- **Pool2:** AvgPool 2×2 , stride 2 $\rightarrow 32 \times 32 \times 64$
- **Conv3:** 128 filters, 3×3 , stride 1, padding 1 $\rightarrow 32 \times 32 \times 128$
- **Pool3:** MaxPool 2×2 , stride 2 $\rightarrow 16 \times 16 \times 128$

Verification Statement (example): If diagram shows Pool1 output as $64 \times 64 \times 32$, this is correct. If it shows different dimensions, state the discrepancy and expected values.

Component	Full (5)	Poor (0-2)
Conv1 formula + calculation (2 marks)	Correct $128 \times 128 \times 32$ with steps	Wrong/missing
Remaining layer verification (3 marks)	Correct dimensions stated for all layers	Multiple errors

Keywords: "Output formula", " $128 \times 128 \times 32$ ", " $64 \times 64 \times 32$ ", "pooling halves dimensions", "depth = number of filters"

Part (ii): Parameter Calculation [5 marks]

Convolutional Layer Parameter Formula:

$$\text{Parameters} = (\text{Kernel size} \times \text{Input channels} \times \text{Output channels}) + \text{Biases}$$

Conv Layer 2 Detailed Calculation (2 marks):

Assume: Input $64 \times 64 \times 32$, 64 filters, kernel 3×3 , stride 1, padding 1

Weights:

$$\begin{aligned} \text{Weight parameters} &= \text{Kernel height} \times \text{Kernel width} \times \text{Input channels} \times \text{Output filters} \\ &= 3 \times 3 \times 32 \times 64 = 18,432 \end{aligned}$$

Biases:

$$\text{Biases} = \text{Number of filters} = 64$$

Total Conv2 Parameters: $18,432 + 64 = 18,496$

Filter Depth Effect Explanation (3 marks):

How Filter Depth Affects Parameter Count:

- **Linear relationship:** Parameters scale linearly with input channels
- **Example Conv1:** Input channels = 3 (RGB) $\rightarrow$ fewer parameters
- **Example Conv3:** Input channels = 64 $\rightarrow$ more parameters ($64/3 \approx 21\times$ increase)
- **Mathematical relationship:** Doubling input channels doubles parameters (for fixed kernel size and output filters)

Why It Matters:

- Deeper layers typically have more input channels (feature maps from previous layer)
- Parameter count grows significantly in deeper layers even with same kernel size
- Example: Conv1 ($3 \times 3 \times 3 \times 32 = 864$) vs Conv3 ($3 \times 3 \times 64 \times 128 = 73,728$) — $85\times$ difference
- Design consideration: early layers process raw pixels (3 channels), later layers process rich features (64+ channels)

Parameter Efficiency Strategies:

- 1×1 convolutions to reduce channel depth before expensive 3×3 operations
- Depthwise separable convolutions (separate spatial and channel-wise filtering)
- Bottleneck architectures

Component	Full	Poor
Conv2 calculation (2 marks)	Correct 18,496 with formula	Wrong/missing
Filter depth explanation (3 marks)	Clear linear relationship with examples	Vague

Keywords: "Parameters = kernel $\times$ input channels $\times$ output channels", "linear scaling", "deeper layers more parameters", "channel depth impact"

(b) Architecture Design and Analysis [10 marks]

Part (i): Flatten Compatibility and FC Parameters [6 marks]

Flatten Compatibility Analysis (3 marks):

Expected Flattened Vector Size:

From Pool3 output: $16 \times 16 \times 128$

$$\text{Flattened size} = 16 \times 16 \times 128 = 32,768$$

FC Layer Input Dimension:

From architecture diagram (assumed): FC layer expects input dimension = 32,768

Compatibility Check:

- Flattened vector: 32,768 elements
- FC input dimension: 32,768 neurons
- **Result: COMPATIBLE**
- If mismatch exists (e.g., diagram shows FC input = 16,384), calculate discrepancy and identify error in architecture

Calculation Justification:

$$\text{Pool3 output} = 16 \times 16 \times 128$$

$$\text{After flatten} = 16 \cdot 16 \cdot 128 = 32,768 \text{ (1D vector)}$$

Must match FC layer's first dimension exactly for matrix multiplication compatibility.

FC Layer Parameter Count (3 marks):

Assume architecture: Flatten $\rightarrow$ FC(32,768 $\rightarrow$ 256) $\rightarrow$ FC(256 $\rightarrow$ 4)

FC Layer 1 (Flatten to Hidden):

$$\text{Weights} = \text{Input size} \times \text{Output size} = 32,768 \times 256 = 8,388,608$$

$$\text{Biases} = 256$$

$$\text{Total FC1} = 8,388,608 + 256 = 8,388,864$$

FC Layer 2 (Hidden to Output):

$$\text{Weights} = 256 \times 4 = 1,024$$

$$\text{Biases} = 4$$

$$\text{Total FC2} = 1,024 + 4 = 1,028$$

Total FC Parameters: $8,388,864 + 1,028 = 8,389,892$

Observation: FC layers dominate parameter count (99

Component	Full	Partial	Poor
Flatten compatibility (3 marks)	Correct 32,768 with verification	Calculation error	Wrong
FC parameters (3 marks)	Both FC layers correct	One correct	Wrong

Keywords: "Flatten 32,768", "compatible", "FC weights = input $\times$ output", "millions of parameters", "FC dominates"

Part (ii): Stride and Pooling Design Choices [4 marks]

Why Stride > 1 in Convolutions (3 marks):

Stride Comparison (2 marks):

Stride = 2 Convolution:

- Combines feature extraction and spatial downsampling in single operation
- Parameters: $3 \times 3 \times C_{in} \times C_{out}$ (learned)
- Learnable downsampling — adapts to data
- More computationally efficient than separate conv + pool

Stride = 1 Conv + MaxPool:

- Separate operations: extract features, then downsample
- Conv parameters: $3 \times 3 \times C_{in} \times C_{out}$
- Pool parameters: 0 (fixed operation)
- Two-step process — more computation

Parameter Efficiency (1 mark):

Example calculation (Conv with 64 input, 128 output channels):

- **Stride-2 Conv:** $3 \times 3 \times 64 \times 128 = 73,728$ params, single pass
- **Stride-1 Conv + Pool:** $3 \times 3 \times 64 \times 128 = 73,728$ params + 0 pool params, two passes
- **Parameter count same** but stride-2 requires fewer operations
- Stride-2 processes fewer spatial locations (halved) — roughly $2\times$ faster
- Memory footprint lower (intermediate feature maps smaller)

For Defect Detection:

- Real-time inspection requires speed — stride > 1 reduces computation
- Learnable downsampling can preserve defect-critical features better than fixed pooling
- Trade-off: may lose fine spatial details (acceptable for larger defects)

AvgPool vs MaxPool Design Choice (3 marks):

Context: Architecture uses AvgPool in Pool2 (after Conv2)

MaxPool Characteristics:

- Selects maximum activation in window
- Emphasizes strong features (edges, corners)
- Discards weak signals
- Translation invariant to small shifts
- Good for: sharp features, edges, high-contrast patterns

AvgPool Characteristics:

- Averages all activations in window
- Preserves subtle patterns and textures
- Retains weak but distributed signals
- Smoother feature maps
- Good for: textures, gradual variations, noise reduction

Why AvgPool for Defect Detection (2 reasons):

Reason 1: Preserve Subtle Patterns (1.5 marks)

- Defects like scratches may have subtle texture variations
- MaxPool discards weak activations — might miss fine scratches
- AvgPool retains distributed information across spatial region
- Critical for detecting low-contrast defects on metal surfaces
- Texture-based defects (surface roughness) benefit from averaging

Reason 2: Noise Robustness (1.5 marks)

- Manufacturing environments have sensor noise, lighting variations
- MaxPool amplifies noise (selects maximum, even if noise spike)
- AvgPool smooths noise through averaging — more robust
- Metal surfaces reflective — specular highlights create noise
- Averaging reduces false positives from reflections

Alternative Acceptable Reasons:

- Balance edge and texture: Pool1 MaxPool (edges), Pool2 AvgPool (textures)
- Reduce overfitting: AvgPool provides smoother, more generalized features
- Better gradient flow: AvgPool distributes gradients to all inputs, not just max

Component	Full	Partial	Poor
Stride justification (2 marks)	Clear comparison with efficiency	Generic statement	Vague
Stride parameter efficiency (1 mark)	Calculation or clear reasoning	Incomplete	Wrong
AvgPool two reasons (3 marks)	Both reasons clear with defect context	One reason clear	Vague

Keywords: "Stride-2 downsamples", "parameter efficient", "fewer operations", "AvgPool subtle patterns", "texture preservation", "noise robustness", "MaxPool edges"

Question 4: RNN Computation and Comparison

(a) RNN/GRU Parameter Counting and Model Selection [7 marks]

Part (i): Parameter Count Computation [2 marks]

Given: $d = 10$ (input dimension), $h = 20$ (hidden dimension)

Vanilla RNN Parameters (1 mark):

- W_{hx} : $h \times d = 20 \times 10 = 200$
- W_{hh} : $h \times h = 20 \times 20 = 400$
- b_h : $h = 20$
- **Total: 620 parameters**

GRU Parameters (1 mark):

- GRU has 3 gates: update (z), reset (r), candidate ($\tilde{h}$)
- Each gate: W_{hx} (200) + W_{hh} (400) + b_h (20) = 620
- **Total: $3 \times 620 = 1860$ parameters**

Component	Full (2)	Partial (1)	None (0)
RNN count (1 mark)	620 with breakdown	Correct formula, calculation error	Wrong
GRU count (1 mark)	1860 with 3 gates explained	Correct approach, minor error	Wrong

Keywords: " W_{hx} , W_{hh} , bias", "620", "3 gates", "1860"

Part (ii): Model Selection for Edge Device [5 marks]

Scenario: Factory edge device - limited memory, compute, power; high-frequency sensor data; recent pattern anomalies

Recommendation: GRU

Justification:

1. Memory Constraints (1.5 marks):

- GRU: 1860 parameters vs LSTM: 2480 parameters (4 gates)
- 25% fewer parameters than LSTM—critical for edge deployment
- Lower memory footprint for model storage and inference

2. Computational Efficiency (1.5 marks):

- GRU: 3 gates (update, reset, candidate) vs LSTM: 4 gates (input, forget, output, cell)
- Fewer matrix multiplications per timestep—faster inference
- High-frequency sensor data requires real-time processing; GRU meets latency requirements

3. Power/Battery Constraints (1.5 marks):

- Fewer computations $\rightarrow$ lower power consumption
- Edge devices run on limited battery—GRU extends operational time
- 25% computational savings directly translates to energy savings

4. Task Suitability (0.5 marks):

- Anomalies caused by recent patterns—don't need extreme long-term memory
- GRU's simpler gating sufficient for short-to-medium dependencies
- LSTM's separate cell state overkill for this task

Alternative Consideration: Vanilla RNN even more efficient (620 params) but would suffer vanishing gradients; GRU balances efficiency with gradient flow.

Component	Excellent (5)	Poor (0-2)
Correct choice (GRU) with 3 constraint justifications (5 marks)	All 3 constraints addressed with factory context	Vague or missing constraints

Breakdown by Constraint:

- Memory: 1.5 marks (parameter comparison, edge deployment context)
- Compute: 1.5 marks (gate comparison, real-time requirement)
- Power: 1.5 marks (energy efficiency, battery life)
- Task fit: 0.5 marks (recent patterns, dependency length)

Keywords: "GRU", "fewer parameters", "3 gates vs 4", "edge device", "battery", "real-time", "recent patterns"

(b) RNN and GRU Numerical Computation [8 marks]**Part (i): RNN and GRU Computation [5 marks]**

Given: $\mathbf{x}_t = [2, 1]^\top$, $\mathbf{h}_{t-1} = [0.5, -0.5]^\top$, $d = 2$, $h = 2$, biases = 0

Vanilla RNN (2 marks):

$$W_{hh} = \begin{bmatrix} 0.6 & 0 \\ 0 & 0.6 \end{bmatrix}, W_{hx} = \begin{bmatrix} 0.4 & 0 \\ 0 & 0.4 \end{bmatrix}$$

Pre-activation:

$$\begin{aligned} W_{hh}\mathbf{h}_{t-1} &= \begin{bmatrix} 0.6 & 0 \\ 0 & 0.6 \end{bmatrix} \begin{bmatrix} 0.5 \\ -0.5 \end{bmatrix} = \begin{bmatrix} 0.3 \\ -0.3 \end{bmatrix} \\ W_{hx}\mathbf{x}_t &= \begin{bmatrix} 0.4 & 0 \\ 0 & 0.4 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.8 \\ 0.4 \end{bmatrix} \\ z &= \begin{bmatrix} 0.3 \\ -0.3 \end{bmatrix} + \begin{bmatrix} 0.8 \\ 0.4 \end{bmatrix} = \begin{bmatrix} 1.1 \\ 0.1 \end{bmatrix} \end{aligned}$$

Hidden state:

$$\mathbf{h}_t = \tanh(z) \approx \begin{bmatrix} \tanh(1.1) \\ \tanh(0.1) \end{bmatrix} \approx \begin{bmatrix} 0.80 \\ 0.10 \end{bmatrix}$$

GRU (3 marks):

$W_z = I$, $U_z = 0$, $W_h = 0.5I$, $U_h = 0$

Using: $\sigma(2) \approx 0.88$, $\sigma(1) \approx 0.73$, $\tanh(1) \approx 0.76$, $\tanh(0.5) \approx 0.46$

Update gate:

$$\mathbf{z}_t = \sigma(W_z\mathbf{x}_t) = \sigma\left(\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix}\right) = \sigma\left(\begin{bmatrix} 2 \\ 1 \end{bmatrix}\right) \approx \begin{bmatrix} 0.88 \\ 0.73 \end{bmatrix}$$

Candidate state:

$$\tilde{\mathbf{h}}_t = \tanh(W_h\mathbf{x}_t) = \tanh\left(\begin{bmatrix} 0.5 & 0 \\ 0 & 0.5 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix}\right) = \tanh\left(\begin{bmatrix} 1 \\ 0.5 \end{bmatrix}\right) \approx \begin{bmatrix} 0.76 \\ 0.46 \end{bmatrix}$$

Final hidden state:

$$\begin{aligned} \mathbf{h}_t &= \mathbf{z}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{z}_t) \odot \tilde{\mathbf{h}}_t \\ &= \begin{bmatrix} 0.88 \\ 0.73 \end{bmatrix} \odot \begin{bmatrix} 0.5 \\ -0.5 \end{bmatrix} + \begin{bmatrix} 0.12 \\ 0.27 \end{bmatrix} \odot \begin{bmatrix} 0.76 \\ 0.46 \end{bmatrix} \approx \begin{bmatrix} 0.531 \\ -0.241 \end{bmatrix} \end{aligned}$$

Keywords: "Pre-activation", "tanh", "update gate", "candidate state", " $\odot$ element-wise"

Part (ii): Conservative Update Analysis [2 marks]**Numerical Reason:**

- GRU update gate: $\mathbf{z}_t = [0.88, 0.73]$ (both close to 1)

Component	Full	Partial	None
RNN computation (2 marks)	All steps, correct $\mathbf{h}_t$	Approach correct, minor error	Wrong
GRU gates (1.5 marks)	$\mathbf{z}_t, \tilde{\mathbf{h}}_t$ correct	One correct	Wrong
GRU final state (1.5 marks)	Correct combination	Formula right, calc error	Wrong

- GRU update: $\mathbf{h}_t = \mathbf{z}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{z}_t) \odot \tilde{\mathbf{h}}_t$
- Retains 88% and 73% of previous state, adds only 12% and 27% new candidate
- **Conservative:** mostly preserves past, minimal new information integration

Vanilla RNN Comparison:

- RNN: $\mathbf{h}_t = \tanh(\dots)$ completely overwrites previous state
- No gating mechanism—replaces old state entirely with new computation
- GRU selectively retains history via high $\mathbf{z}_t$ values

Component	Full (2)	Poor (0-1)
Numerical comparison with update gate values (2 marks)	Clear explanation using $\mathbf{z}_t$ close to 1	Vague or missing

Keywords: " $\mathbf{z}_t$ close to 1", "retains previous state", "small new contribution", "RNN overwrites"

Part (iii): BPTT and RNN vs Transformer [1 mark]

****Note:**** This part was marked as 3 marks in the delta but the question structure shows it should fit within the 8-mark total for part (b). Adjusting to 1 mark to maintain total.

BPTT Gradient:

Given: $\mathbf{h}_t = z_t \mathbf{h}_{t-1} + (1 - z_t) \tilde{\mathbf{h}}_t$, candidate path ignored

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} \approx z_t = 0.9$$

Backpropagating from $t = 3$ to $t = 1$:

$$\begin{aligned} \frac{\partial L}{\partial \mathbf{h}_2} &\approx \frac{\partial L}{\partial \mathbf{h}_3} \cdot z_3 = 1 \times 0.9 = 0.9 \\ \frac{\partial L}{\partial \mathbf{h}_1} &\approx \frac{\partial L}{\partial \mathbf{h}_2} \cdot z_2 = 0.9 \times 0.9 = 0.81 \end{aligned}$$

Answer: $\partial L / \partial \mathbf{h}_1 \approx 0.81$

Component	Full (1)	None (0)
Gradient computation (1 mark)	Correct 0.81 with steps	Wrong/missing

Keywords: "0.81", "multiply 0.9 twice", "backpropagate"

Question 5: Attention Mechanisms

(a) Scaled Dot-Product Attention and Masked Attention [7 marks]

Part (i): Attention Computation [3 marks]

Given: $\mathbf{q}_3 = [2, 7]$, keys $\mathbf{k}_1 = [3, 1]$, $\mathbf{k}_2 = [3, 1]$, $\mathbf{k}_3 = [7, 3]$, values $\mathbf{v}_1 = [3, 1]$, $\mathbf{v}_2 = [3, 1]$, $\mathbf{v}_3 = [8, 3]$

Step 1: Compute raw scores

$$\text{score}_1 = \mathbf{q}_3 \cdot \mathbf{k}_1 = (2)(3) + (7)(1) = 6 + 7 = 13$$

$$\text{score}_2 = \mathbf{q}_3 \cdot \mathbf{k}_2 = (2)(3) + (7)(1) = 13$$

$$\text{score}_3 = \mathbf{q}_3 \cdot \mathbf{k}_3 = (2)(7) + (7)(3) = 14 + 21 = 35$$

Step 2: Scale scores

$$d_k = 2, \quad \sqrt{d_k} = \sqrt{2} \approx 1.414$$

$$\text{scaled}_1 = 13/1.414 \approx 9.19$$

$$\text{scaled}_2 = 13/1.414 \approx 9.19$$

$$\text{scaled}_3 = 35/1.414 \approx 24.75$$

Step 3: Apply softmax

$$\alpha_1 \approx 0, \quad \alpha_2 \approx 0, \quad \alpha_3 \approx 1$$

Step 4: Compute context output

$$\text{context} = \alpha_1 \mathbf{v}_1 + \alpha_2 \mathbf{v}_2 + \alpha_3 \mathbf{v}_3 \approx (0)[3, 1] + (0)[3, 1] + (1)[8, 3] = [8, 3]$$

Interpretation: Model attends almost entirely to Headset (position 3), ignoring past items Phones and Bags.

Component	Full (3)	Partial (1.5-2)	Poor (0-1)
Dot products (1 mark)	All correct: 13, 13, 35	One error	Multiple errors
Scaling + softmax (1 mark)	Correct weights $\approx [0,0,1]$	Scaling correct, softmax error	Wrong
Context vector (1 mark)	Correct [8,3]	Formula right, calc error	Wrong

Keywords: "Dot product", "scale by $\sqrt{d_k}$ ", "softmax", "weighted sum", "[8,3]"

Part (ii): Missing Attention Mechanism [4 marks]

Problem: Model recommends what users bought in training but fails on future purchases in production.

Missing Mechanism: Masked (Causal) Attention

Explanation:

- During training without masking, attention at position 3 (Headset) can see future information (what user actually bought next)
- Model learns to "cheat"—simply copying training patterns rather than learning predictive relationships
- At inference (production), future purchases don't exist—model cannot use its learned strategy
- Result: perfect training performance, catastrophic production failure

Solution:

- Apply causal masking during training: position i cannot attend to positions $j > i$
- Forces model to predict next item using only past browsing history
- Training conditions match production conditions—model learns true predictive patterns

Mathematical Implementation: Set attention scores to $-\infty$ for future positions before softmax, ensuring zero attention weight.

Component	Full (4)	Poor (0-2)
Identifies masked/causal attention (2 marks)	Correct with clear reasoning	Vague or wrong
Explains train-test mismatch (2 marks)	Clear explanation of information leakage	Incomplete

Keywords: "Masked attention", "causal attention", "future information leakage", "training cheating", "production mismatch"

(b) Additive Attention and Multi-Head Analysis [8 marks]**Part (i): Additive Attention and Comparison [5 marks]**

Given: $\mathbf{q}_3 = [2, 1]$, keys $\mathbf{k}_1 = [1, 0]$, $\mathbf{k}_2 = [0, 1]$, $\mathbf{k}_3 = [2, 1]$, values $\mathbf{v}_1 = [1, 0]$, $\mathbf{v}_2 = [1, 1]$, $\mathbf{v}_3 = [3, 1]$

Additive Attention Computation (3 marks):

Assume weight vectors: $\mathbf{w}_q = [1, 1]$, $\mathbf{w}_k = [1, 1]$, $\mathbf{v} = [1, 1]$ (standard settings)

Step 1: Compute alignment scores

$$\begin{aligned}
 e_1 &= \mathbf{v}^\top \tanh(\mathbf{w}_q \mathbf{q}_3 + \mathbf{w}_k \mathbf{k}_1) \\
 &= [1, 1]^\top \tanh([1, 1][2, 1] + [1, 1][1, 0]) \approx 1.756 \\
 e_2 &= [1, 1]^\top \tanh([3] + [1]) = [1, 1]^\top \tanh([2, 2]) \approx 1.928 \\
 e_3 &= [1, 1]^\top \tanh([3] + [3]) = [1, 1]^\top \tanh([4, 2]) \approx 1.963
 \end{aligned}$$

Step 2: Apply softmax

$$\alpha_1 \approx 0.292, \quad \alpha_2 \approx 0.348, \quad \alpha_3 \approx 0.360$$

Step 3: Context output

$$\text{context} = 0.292[1, 0] + 0.348[1, 1] + 0.360[3, 1] \approx [1.72, 0.708]$$

Interpretation: X-ray (position 2) receives more attention than Blood Test, reflecting its greater diagnostic relevance.

Why Additive vs Scaled Dot-Product (2 marks):**Additive Attention Advantages:**

- More expressive—learnable weight matrices $\mathbf{w}_q$, $\mathbf{w}_k$, $\mathbf{v}$ provide additional parameters
- Better for low-dimensional embeddings (like medical events with $d = 2$)
- Can learn complex non-linear score functions via $\tanh$

Scaled Dot-Product Advantages:

- Faster—no $\tanh$ computation, just matrix multiplication
- Scales better to high dimensions and large vocabularies
- Parallelizes efficiently on GPUs

Component	Full	Partial	Poor
Additive computation (3 marks)	All steps, correct context	Approach right, calc error	Wrong
Comparison (2 marks)	Both mechanisms compared with EHR justification	Generic comparison	Vague

For EHR Scenario: Additive attention preferred—medical events have rich semantic meaning requiring expressive scoring, low-dimensional space benefits from learnable transformations, interpretability critical for clinical decisions.

Keywords: "tanh", "alignment scores", "learnable weights", "expressiveness vs speed", "low-dimensional"

Part (ii): Multi-Head Attention Analysis [3 marks]

Given: $d_{model} = 512$, $h = 8$ heads

Head Dimension Calculation (1 mark):

$$d_k = d_v = \frac{d_{model}}{h} = \frac{512}{8} = 64$$

Each head's Q/K/V projections: dimension 64

Parameter Comparison (1 mark):

Single-head attention:

- $W_Q: 512 \times 512$ $W_K: 512 \times 512$ $W_V: 512 \times 512$
- Total: $3 \times 512^2 = 786,432$ parameters

8-head attention:

- Per head: W_Q, W_K, W_V each 512×64
- 8 heads: $8 \times 3 \times 512 \times 64 = 786,432$
- Output projection $W_O: 512 \times 512 = 262,144$
- Total: $786,432 + 262,144 = 1,048,576$ parameters

Comparison: Multi-head has 33% more parameters due to output projection.

Multi-Head Resolution and Limitation (1 mark):

How Multi-Head Resolves Single-Head Problem:

- Different heads learn different attention patterns simultaneously
- Head 1: recent events (temporal recency)
- Head 2: symptom-medication correlations
- Head 3: lab result trends
- Head 4: imaging-diagnosis connections
- Captures multiple clinical relationships in parallel—no single pattern dominates

Limitation in Resource-Constrained Deployment:

- 33% more parameters increases memory footprint
- $8\times$ parallel attention computations require more GPU memory
- Higher inference latency on edge devices with limited parallelism
- Mitigation: reduce heads (e.g., 4 heads) or use smaller d_{model}

Component	Full (3)	Poor (0-1)
Head dimension (1 mark)	64 with calculation	Wrong
Parameter comparison (1 mark)	Correct counts for both	Wrong
Multi-head benefit + limitation (1 mark)	Clear clinical examples + deployment constraint	Vague

Keywords: " $d_k = 64$ ", "different heads learn different patterns", "memory footprint", "parallel computation cost"

Question 6: Transformers for Clinical NLP

(a) Positional Encoding and Architecture Matching [10 marks]

Part (i): Positional Encoding Failure Mode [5 marks]

What Positional Encoding Does (2 marks):

- Transformers use self-attention—inherently permutation-invariant (no sequential order awareness)
- Positional encoding injects position information into token embeddings
- Allows model to distinguish "patient reported fever then cough" from "patient reported cough then fever"
- Encodes absolute or relative position of each token in sequence

Common Methods:

- Sinusoidal: $PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{model}})$
- Learned embeddings: trainable position vectors added to token embeddings

Failure Mode Without Positional Encoding (3 marks):

- Clinical notes would be treated as bag-of-words—temporal relationships lost
- "Prescribed medication before lab test" vs "Lab test before prescribed medication" indistinguishable
- Medical reasoning depends critically on temporal sequence:
 - Symptom progression: "fever → cough → difficulty breathing" (worsening) vs "difficulty breathing → fever → cough" (improving)
 - Treatment response: "administered medication → symptoms improved" vs "symptoms improved → administered medication" (different causality)
 - Diagnostic timeline: "blood pressure high → medication started → blood pressure normal" loses causal chain
- Model cannot learn temporal patterns, dependencies, or causal relationships
- Summarization would scramble event order—clinically dangerous

Alternative Acceptable Explanations:

- All permutations of same tokens treated identically
- Cannot distinguish early vs late symptoms
- Loses procedural order (test → diagnosis → treatment)

Component	Full (5)	Poor (0-2)
What PE does (2 marks)	Clear explanation with position injection	Vague
Clinical failure mode (3 marks)	Temporal relationships lost with medical examples	Generic or missing

Keywords: "Permutation invariant", "position information", "sequential order", "temporal relationships", "bag-of-words", "clinical timeline"

Part (ii): Architecture Matching [5 marks]

Three Architectures:

- **Encoder-only:** Bidirectional self-attention on input sequence (e.g., BERT)
- **Decoder-only:** Causal (masked) self-attention for autoregressive generation (e.g., GPT)
- **Encoder-decoder:** Encoder processes input, decoder generates output with cross-attention (e.g., T5, BART)

Task Matching (5 marks):

(a) Clinical Named Entity Recognition → Encoder-Only (1.5 marks)

- **Task:** Tag each token (medication, symptom, dosage, diagnosis)
- **Why encoder-only:**

- Bidirectional context needed—"prescribed 500mg aspirin daily" requires seeing "500mg" and "daily" to tag "aspirin"
- No generation required—input and output same length
- Each token classification benefits from full sentence context
- **Architecture:** BERT-style with token classification head
- (b) **Discharge Note Generation → Decoder-Only (1.5 marks)**
- **Task:** Generate discharge summary from scratch or with minimal prompt
- **Why decoder-only:**
 - Autoregressive generation—predicts next word given previous words
 - Causal masking ensures coherent left-to-right text generation
 - No fixed input sequence to encode
 - Suitable for open-ended text generation
- **Architecture:** GPT-style with language modeling head
- (c) **Medical Report Summarization → Encoder-Decoder (2 marks)**
- **Task:** Read full discharge note (input), generate concise summary (output)
- **Why encoder-decoder:**
 - Encoder: bidirectionally processes entire input document
 - Decoder: autoregressively generates summary
 - Cross-attention: decoder attends to relevant input parts while generating each summary token
 - Input and output different lengths—compression required
 - Explicit input-output mapping needed (not open generation)
- **Architecture:** T5-style or BART with seq2seq setup

Task	Correct Match	Strong Justification	Weak/Wrong
NER (1.5 marks)	Encoder-only (0.5)	Bidirectional context need (1)	Missing reasoning
Generation (1.5 marks)	Decoder-only (0.5)	Autoregressive explanation (1)	Missing reasoning
Summarization (2 marks)	Encoder-decoder (0.5)	Input-output mapping + cross-attention (1.5)	Missing reasoning

Keywords: "Encoder-only", "bidirectional", "decoder-only", "causal/autoregressive", "encoder-decoder", "cross-attention", "seq2seq"

(b) Transformer Encoder Analysis and Variants [10 marks]

Part (i): Encoder Layer Analysis [6 marks]

Given: $d_{model} = 512$, $h = 8$ heads, $d_{ff} = 2048$, $L = 128$ tokens

Dimension Tracing (3 marks):

Input: $(L, d_{model}) = (128, 512)$

Multi-head attention output:

- Per head: Q, K, V each $(128, 64)$ where $d_k = 512/8 = 64$
- Attention per head: $(128, 64)$
- Concatenate 8 heads: $(128, 512)$
- After output projection W_O : $(128, 512)$

Add & Norm 1: $(128, 512)$ (residual + layer norm, preserves dimensions)

Feed-forward output:

- First linear: $(128, 512) \rightarrow (128, 2048)$
 - Activation (ReLU/GELU): $(128, 2048)$
 - Second linear: $(128, 2048) \rightarrow (128, 512)$
- Add & Norm 2:** $(128, 512)$ (final output)

Parameter Count (3 marks):

Multi-head attention:

- W_Q, W_K, W_V : $3 \times 512 \times 512 = 786,432$
- Output projection W_O : $512 \times 512 = 262,144$
- Biases (Q, K, V, O): $4 \times 512 = 2,048$
- **Subtotal: 1,050,624 parameters**

Feed-forward network:

- First linear: $512 \times 2048 + 2048 = 1,050,624$
- Second linear: $2048 \times 512 + 512 = 1,049,088$
- **Subtotal: 2,099,712 parameters**

Layer normalization:

- Two LayerNorm (scale + shift): $2 \times 2 \times 512 = 2,048$

Total per encoder layer: $1,050,624 + 2,099,712 + 2,048 = 3,152,384$ **parameters**

Component	Full	Partial	Poor
Dimension trace (3 marks)	All stages correct	1-2 errors	Multiple errors
Attention params (1.5 marks)	Correct 1.05M	Approach right, calc error	Wrong
FFN params (1.5 marks)	Correct 2.1M	Approach right, calc error	Wrong

Keywords: "(128,512)", "multi-head concat", "residual connection", " $d_{ff} = 2048$ ", " 3.15M parameters per layer"

Part (ii): Transformer Variants and Complexity [4 marks]

BERT, GPT, T5 Comparison (2 marks):

Architecture	Pre-training	Form	Summarization Fit
BERT-style	Masked language modeling (predict masked tokens)	Encoder-only	Poor—no generation capability
GPT-style	Causal language modeling (next token prediction)	Decoder-only	Moderate—can generate but no input encoding
T5-style	Span corruption (predict masked spans)	Encoder-decoder	Excellent—designed for seq2seq tasks

Recommendation for Summarization: T5-style

- Encoder processes full input note bidirectionally
- Decoder generates summary autoregressively
- Cross-attention links generated summary to source document
- Pre-trained on diverse text-to-text tasks including summarization

Computational Bottleneck and Mitigation (2 marks):

Bottleneck:

- Self-attention complexity: $O(L^2 \cdot d_{model})$ where $L = 2048$ tokens
- For clinical notes: $2048^2 = 4,194,304$ pairwise token comparisons

- Memory: storing attention matrix (2048, 2048) per head = 33.5M values (8 heads)
- Computation: quadratic scaling makes long documents prohibitively expensive

Mitigation Strategy - Sliding Window Attention:

- Each token attends only to local window (e.g., $w = 256$ tokens)
- Complexity reduces from $O(L^2)$ to $O(L \cdot w)$
- For $L = 2048$, $w = 256$: $2048 \times 256 = 524,288$ ($8\times$ reduction)

Trade-off:

- **Benefit:** Massive memory and computation savings, enables longer sequences
- **Cost:** Lost long-range dependencies—token at position 0 cannot directly attend to position 2000
- **Partial mitigation:** Combine with strided attention or hierarchical attention for limited long-range connections

Alternative Mitigation Strategies:

- Sparse attention (Longformer): combines local + global attention
- Linformer: low-rank approximation reduces to $O(L \cdot k)$
- Reformer: locality-sensitive hashing for efficient attention

Component	Full (4)	Poor (0-2)
BERT/GPT/T5 comparison (2 marks)	Clear distinction with summarization fit	Incomplete
$O(L^2)$ bottleneck explanation (1 mark)	Clear with 2048 token context	Vague
Mitigation + trade-off (1 mark)	Strategy with benefit and cost	Missing trade-off

Keywords: "BERT encoder-only", "GPT decoder-only", "T5 seq2seq", " $O(L^2)$ complexity", "sliding window", "long-range dependencies"

Question 7: Optimization Strategies

(a) Optimizer Curve Matching [5 marks]

Context: Four optimizers (SGD, SGD+Momentum, RMSprop, Adam) trained over 100 epochs. Training loss curves show distinct patterns.

Curve Characteristics:

- **Curve A:** Smooth but very slow descent, still decreasing at epoch 100
- **Curve B:** Fast initial descent with oscillations, settles near good minimum
- **Curve C:** Fast descent, minimal oscillation, plateaus early at higher loss than D
- **Curve D:** Fastest convergence, smooth descent, reaches lowest loss

Matching and Justification (4 marks):

Curve A → SGD (1 mark)

- Fixed learning rate with no momentum or adaptive scaling
- Slowest convergence—pure gradient direction without acceleration
- Smooth because no momentum oscillations
- Still improving at epoch 100—would eventually converge but requires many epochs

Curve B → SGD + Momentum (1 mark)

- Momentum accelerates convergence (faster than vanilla SGD)
- Oscillations from momentum overshooting—"ball rolling downhill" effect
- Eventually settles as velocity dampens near minimum
- Faster than SGD but less stable than adaptive methods

Curve C → RMSprop (1 mark)

- Adaptive learning rates per parameter—fast initial descent
- Minimal oscillation due to gradient scaling
- Plateaus early at suboptimal loss—can get stuck in local minima
- No momentum component to escape plateaus

Curve D → Adam (1 mark)

- Combines momentum and adaptive learning rates—best of both
- Fastest convergence with smooth trajectory
- Adaptive scaling prevents oscillations
- Momentum helps escape shallow local minima—reaches lowest loss
- Industry standard for deep learning—reliable across diverse tasks

Curve	Optimizer	Full Credit (1)	No Credit (0)
A	SGD	Correct match + slow/smooth justification	Wrong match
B	SGD+Momentum	Correct match + oscillation justification	Wrong match
C	RMSprop	Correct match + plateau justification	Wrong match
D	Adam	Correct match + best performance justification	Wrong match

Keywords: "SGD slow", "momentum oscillations", "RMSprop adaptive", "Adam combines both", "plateau", "convergence speed"

Part (ii): Overfitting Intervention [1 mark]

Problem: After epoch 60 using Adam, validation loss rises while training loss falls.

Phenomenon: Overfitting—model memorizing training data, losing generalization.

Optimizer-Level Intervention: Reduce learning rate (learning rate decay/scheduling)

Mechanism:

- High learning rate causes large weight updates—model overfits to training noise
- Reducing learning rate (e.g., multiply by 0.1 at epoch 60) forces smaller, more conservative updates

- Smaller steps prevent aggressive memorization of training examples
- Allows model to "settle" into flatter, more generalizable minima
- Adam's adaptive rates still active but scaled down globally

Implementation: ReduceLROnPlateau or step decay at epoch 60

Alternative Acceptable Interventions:

- Early stopping—halt training at epoch 60
- Gradient clipping—limit update magnitude (less direct for overfitting)
- Weight decay increase (though borderline architectural)

Component	Full (1)	None (0)
Identifies overfitting + LR reduction with mechanism (1 mark)	Clear explanation	Vague/wrong

Keywords: "Overfitting", "learning rate decay", "smaller updates", "generalization", "ReduceLROnPlateau"

(b) Learning Rate Schedules and Analysis [5 marks]

Part (i): Schedule Type Identification [3 marks]

Context: Three LR schedules on Adam produce different validation accuracy curves over 50 epochs.

Curve Patterns:

- **Schedule X:** Accuracy rises sharply, then drops and oscillates
- **Schedule Y:** Rises steadily, plateaus at epoch 40, no further improvement
- **Schedule Z:** Rises steadily, brief dip at epoch 30, recovers to highest accuracy

Schedule Matching (3 marks):

Schedule X → Constant High LR (1 mark)

- Sharp initial rise—large updates find good region quickly
- Then drops and oscillates—high LR causes overshooting around minimum
- Never stabilizes—continues bouncing, cannot fine-tune
- Classic symptom of learning rate too high for convergence

Schedule Y → Constant Low LR (1 mark)

- Steady gradual rise—small updates progress slowly but safely
- Plateaus at epoch 40—converged to accessible minimum
- No further improvement—insufficient learning capacity to escape, explore better regions
- Safe but suboptimal—never reaches performance potential

Schedule Z → Cosine Annealing with Warm Restarts (1 mark)

- Steady rise initially—normal training progress
- Brief dip at epoch 30—warm restart increases LR temporarily, disrupting current minimum
- Recovers to highest accuracy—restart helped escape local minimum, find better solution
- Periodic LR increases act as "exploration phases"
- Final convergence superior to constant schedules

Schedule	Type	Full (1)	None (0)
X	Constant High LR	Correct + oscillation reasoning	Wrong
Y	Constant Low LR	Correct + plateau reasoning	Wrong
Z	Cosine Annealing	Correct + restart dip reasoning	Wrong

Keywords: "High LR oscillates", "low LR plateaus", "cosine annealing", "warm restart", "dip at restart"

Part (ii): Schedule Z Dip Explanation [2 marks]

Why Dip at Epoch 30 (1 mark):

- Cosine annealing with warm restart temporarily increases learning rate at epoch 30
- Model was in a local minimum—stable but not optimal
- LR increase causes larger weight updates—destabilizes current solution
- Temporary accuracy drop as model "jumps out" of local minimum
- This disruption is intentional—forced exploration

Why Recovery to Higher Accuracy (1 mark):

- After restart, LR decays again via cosine schedule
- Model explores new region of loss landscape with high initial LR
- Gradually fine-tunes into better minimum as LR decreases
- New minimum has lower loss than pre-restart position
- Demonstrates warm restarts enable escaping suboptimal local minima
- Multiple restarts throughout training increase chances of finding global optimum

Mathematical Intuition:

- Cosine schedule: $\eta_t = \eta_{min} + \frac{1}{2}(\eta_{max} - \eta_{min})(1 + \cos(\frac{T_{cur}}{T_{max}}\pi))$
- At restart: T_{cur} resets to 0, LR jumps to η_{max}
- Then decays smoothly back to η_{min} over restart period

Component	Full (2)	Poor (0-1)
Dip explanation + recovery mechanism (2 marks)	Clear explanation of restart dynamics	Vague or incomplete

Keywords: "Warm restart", "LR increase", "escape local minimum", "cosine decay", "exploration then exploitation"

Question 8: Regularization Strategies

(a) Regularization Diagnosis [5 marks]

Context: Four regularization experiments show different training/validation accuracy patterns over 100 epochs.

Observed Patterns:

- **Run 1:** Train 98%, Val 71% — large gap persists
- **Run 2:** Train 84%, Val 83% — small gap, both plateau early
- **Run 3:** Train 91%, Val 90% — small gap, smooth convergence
- **Run 4:** Train 76%, Val 74% — noisy, slow, unstable

Part (i): Diagnosis for Each Run [4 marks]

Run 1 → Overfitting (1 mark)

- Large persistent gap (98% vs 71%) — classic overfitting signature
- High training accuracy indicates model has sufficient capacity
- Poor validation performance shows memorization, not generalization
- Model fits training noise and specific examples rather than underlying patterns
- Needs stronger regularization

Run 2 → Under-regularized / Underfitting (1 mark)

- Small gap (84% vs 83%) — train and validation track closely
- Both plateau early at suboptimal values — insufficient model capacity
- Could also indicate: learning rate too low, stopped too early, or architecture too simple
- Model hasn't learned data complexity — needs more capacity or training
- Alternative interpretation: over-regularized if strong regularization applied

Run 3 → Well-regularized (1 mark)

- Small gap (91% vs 90%) with high absolute performance
- Smooth convergence — stable, healthy training
- Both curves converging together — generalization matches training
- Optimal balance: model complexity, regularization, and training time
- Target configuration for deployment

Run 4 → Over-regularized (1 mark)

- Very small gap (76% vs 74%) but both low — excessive regularization
- Noisy, slow convergence — regularization fighting learning process
- Unstable curves — optimization struggling against regularization constraints
- Model underfitting due to too much regularization penalty
- Needs reduced regularization strength or removal of some techniques

Run	Diagnosis	Full (1)	Poor (0)
Run 1	Overfitting	Correct with gap justification	Wrong/vague
Run 2	Underfitting	Correct with plateau reasoning	Wrong/vague
Run 3	Well-regularized	Correct with convergence analysis	Wrong/vague
Run 4	Over-regularized	Correct with instability explanation	Wrong/vague

Keywords: "Overfitting large gap", "underfitting plateau", "well-regularized small gap high performance", "over-regularized noisy low performance"

Part (ii): Regularization for Run 1 [1 mark]

Problem: Run 1 shows overfitting (Train 98%, Val 71%, persistent gap)

Two Regularization Techniques:

Technique 1: Dropout (0.5 marks)

- **Mechanism:** Randomly drops neurons (set to 0) during training with probability p (e.g., 0.3-0.5)

- During each forward pass, different random subset of neurons active
- Prevents co-adaptation — neurons cannot rely on specific other neurons
- Forces distributed representations across network
- At inference, all neurons active but outputs scaled by $(1 - p)$
- **Reduces gap by:** Prevents memorization of specific training patterns, forces robust features

Technique 2: L2 Regularization / Weight Decay (0.5 marks)

- **Mechanism:** Adds penalty term to loss: $L_{total} = L_{data} + \lambda \sum w_i^2$
- Penalizes large weights — encourages smaller, more distributed weight values
- During gradient descent: $w \leftarrow w - \eta(\nabla L_{data} + 2\lambda w)$
- Weight decay effect: $w \leftarrow (1 - 2\eta\lambda)w - \eta\nabla L_{data}$
- **Reduces gap by:** Limits model complexity, prevents fitting to noise with extreme weight values, encourages simpler decision boundaries

Alternative Acceptable Techniques:

- Data augmentation — artificially increases training data diversity
- Early stopping — halt training when validation performance peaks
- Batch normalization — normalizes activations, implicit regularization
- Reduce model capacity — fewer layers/neurons

Component	Full (1)	Poor (0)
Two techniques with mechanisms (1 mark)	Both clear with gap reduction explanation	Vague/missing mechanism

Keywords: "Dropout random", "co-adaptation", "L2 penalty", "weight decay", "gap reduction"

(b) Batch Normalization and Comparison [5 marks]

Part (i): Batch Normalization Effect on Run 4 [3 marks]

Context: Run 4 shows over-regularization (Train 76%, Val 74%, noisy, slow, unstable)

Predicted Effects:

Training Stability (1.5 marks):

- **Mechanism:** BN normalizes layer inputs: $\hat{x} = \frac{x - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$
- Reduces internal covariate shift — stabilizes input distributions across layers
- Smooths loss landscape — allows larger learning rates without divergence
- Reduces sensitivity to weight initialization
- **Prediction:** Noisy training curves in Run 4 will stabilize significantly
- Oscillations and instability reduced — smoother convergence trajectory
- Can increase learning rate 2-10× without instability

Convergence Speed (1.5 marks):

- **Mechanism:** Normalized activations prevent vanishing/exploding gradients
- Gradients flow more uniformly through network layers
- Higher learning rates possible due to stability — faster parameter updates
- Acts as mild regularizer but primarily accelerates training
- **Prediction:** Run 4's slow convergence (currently 76% at epoch 100) will accelerate
- Expect 3-5× faster convergence — reaching similar performance in 20-30 epochs
- May reach higher final accuracy (80-85%) due to better optimization

Overall Impact on Run 4:

- BN addresses optimization issues, not regularization level
- Will improve training dynamics but Run 4 still over-regularized
- May need to reduce other regularization (dropout strength, weight decay) alongside BN

- BN itself provides implicit regularization — further complicates over-regularization

Component	Full (3)	Poor (0-1)
Stability prediction + mechanism (1.5 marks)	Clear normalization effect on noise	Vague
Speed prediction + mechanism (1.5 marks)	Clear gradient flow explanation	Vague

Keywords: "Normalize activations", "internal covariate shift", "smooth loss landscape", "faster convergence", "gradient flow"

Part (ii): Dropout vs Batch Normalization [2 marks]

When to Use Each (1 mark):

Dropout:

- **When:** Clear overfitting (large train-val gap), fully connected layers, need strong regularization
- **Where:** Between FC layers in classifier heads, not in convolutional layers (spatial dropout variant exists)
- **Example:** Run 1 scenario — high capacity model memorizing training data

Batch Normalization:

- **When:** Training instability, slow convergence, deep networks, want to use higher learning rates
- **Where:** After conv/linear layers, before activation (or after, debated)
- **Example:** Run 4 scenario — unstable training curves

Can They Be Combined? (1 mark)

Yes, commonly combined:

- **Complementary roles:** BN for optimization/stability, Dropout for regularization
- **Standard pattern:** Conv → BN → ReLU → Dropout (in CNNs)
- **Placement matters:** BN before activation, Dropout after activation
- **Caution:** BN provides implicit regularization — may need lower Dropout rate (0.2-0.3 instead of 0.5)

Architectural Reasoning:

- Modern architectures (ResNet, EfficientNet) use BN throughout backbone
- Dropout typically only in final classifier layers
- Both together addresses different failure modes: instability (BN) and overfitting (Dropout)
- BN's batch statistics during training create stochasticity — weak regularization effect
- Combining maximizes both stability and generalization

Alternative Acceptable Answers:

- Layer Normalization as BN alternative (better for small batches, RNNs)
- DropConnect instead of Dropout (drops weights not activations)
- When not to combine: very small batches (BN unstable), inference-critical applications (Dropout adds randomness)

Component	Full (2)	Poor (0-1)
When to use each (1 mark)	Clear distinction with scenarios	Vague
Combination reasoning (1 mark)	Explains complementary roles	Missing justification

Keywords: "Dropout for overfitting", "BN for stability", "complementary", "combine in same network", "different roles"